Nihara D Aura — Final Web PRD

Customer-Facing Website • No Admin Panel • APK-Based Recreation

Purpose: Build a production-ready web version of the uploaded Nihara D Aura Android application. The APK is the primary product/UX reference. Preserve the customer-facing business functionality while adapting the experience naturally for web.

Critical constraint: The entire admin component must be removed. Do not create an admin panel, admin routes, admin login, admin APIs, admin CRUD screens, or admin-only UI.

1. Product Vision

- Build a premium, elegant beauty and lifestyle e-commerce + salon/service-booking website for Nihara D Aura.
- Use the APK as the source of truth for visible customer flows, terminology, product/service concepts, and overall visual direction.
- Do not make a literal mobile-app clone. Create a responsive, web-native experience with proper desktop navigation, grids, product pages, booking flows, and checkout.

2. Customer Features

- Authentication: register, login, OTP verification, password creation/reset, logout, protected customer pages.
- Home: premium hero area, product discovery, collections, best sellers, makeup/products section, signature salon services, and booking CTA.
- Products: product listing, categories, search, filters, sorting, pagination, product detail, pricing, stock, images, specifications, ratings/reviews where available.
- Cart and checkout: add/remove/update quantities, address selection, order summary, discount/coupon support, payment initiation and payment verification.
- Wishlist: add/remove products and view saved products.
- Beauty services: browse services, view service details, choose artist/specialist, choose salon or at-home service, select date/time, provide address for home service, review booking summary, confirm booking.
- Orders: order history, order detail, payment/order status.
- Bookings: booking history, booking detail, booking status.
- Profile: view/edit profile, saved addresses, account information.

3. APK Reference — Observed Information

- Flutter-based Android application with customer flows covering shopping and beauty/salon booking.
- Observed route concepts include home, products, product detail, categories, cart, checkout, wishlist, profile, orders, bookings, addresses, login/register, onboarding/splash, and booking.
- Observed product categories include Makeup & Cosmetics, Jewellery, Nails, and Home Fragrance.
- Observed examples include foundations, lipsticks, nail products, jewellery, candles, wax melts, diffusers, and room fragrances.
- Observed service categories include makeup, hair, nails, jewellery/styling, and related salon/beauty services.
- Observed booking choices include artist/specialist selection, At Salon vs At Home, date, time, address for home service, and booking confirmation.

- Observed salon reference: Nihara Flagship Salon / Bandra, Mumbai.
- Observed copy/theme includes premium phrases around timeless elegance, Nihara products/collections, makeup services, signature salon services, and expert artists.
- APK references an existing backend and also contains mock/preloaded product references. Do not blindly depend on the APK backend for production.

4. Required Routes

- **Public:** /, /products, /products/:id, /categories, /categories/:slug, /services, /services/:id, /login, /register, /verify-otp
- **Customer:** /cart, /checkout, /wishlist, /profile, /profile/edit, /orders, /orders/:id, /bookings, /bookings/:id, /addresses, /book
- **Forbidden:** No /admin route and no admin equivalent under another URL.

5. Frontend Technology

- React.js + Vite
- Tailwind CSS
- React Router
- Axios
- Recommended where useful: React Hook Form, Zod, Lucide React, TanStack Query
- Use reusable components and a clean feature-oriented architecture.

6. Backend Technology

- Node.js + Express.js
- MongoDB + Mongoose
- JWT-based authentication with secure session/cookie handling where appropriate
- Cloudinary for product/service/user images if required
- Razorpay or another suitable payment gateway for payments
- OTP/email provider for verification and account recovery
- Use a service/API adapter layer so the frontend is not tightly coupled to a specific backend implementation.

7. Database Collections

- users, products, categories, services, artists, salons, orders, bookings, addresses, wishlists, carts, reviews, coupons, payments, otpVerifications
- Product fields should support name, slug, descriptions, images, price, discount price, category, subcategory, stock, SKU, rating, review count, specifications, featured/bestseller flags, active state, and timestamps.
- Service fields should support name, slug, description, category, images, price, duration, salon/home availability, travel charge, available artists, active state, and timestamps.
- Booking fields should support user, services, artist, salon, service type, address, booking date, time slot, duration, subtotal, travel charge, total, payment status, booking status, notes, and timestamps.
- Order fields should support user, items, shipping address, subtotal, discount, delivery charge, total, payment status, order status, payment ID, and timestamps.

8. API Requirements

- Auth: POST /api/auth/register, /login, /verify-otp, /create-password, /logout, /forgot-password, /reset-password
- Products: GET /api/products, GET /api/products/:id
- Categories: GET /api/categories, GET /api/categories/:id
- Cart: GET /api/cart, POST /api/cart, PUT /api/cart/:productId, DELETE /api/cart/:productId
- Wishlist: GET /api/wishlist, POST /api/wishlist/:productId, DELETE /api/wishlist/:productId
- Orders: GET /api/orders, GET /api/orders/:id, POST /api/orders
- Services: GET /api/services, GET /api/services/:id
- Artists: GET /api/artists, GET /api/artists/:id
- Bookings: GET /api/bookings, GET /api/bookings/:id, POST /api/bookings, DELETE /api/bookings/:id
- Profile: GET /api/profile, PUT /api/profile/update
- Addresses: GET /api/addresses, POST /api/addresses, PUT /api/addresses/:id, DELETE /api/addresses/:id

9. Frontend Structure

- src/assets
- src/components — Navbar, Footer, ProductCard, ServiceCard, ArtistCard, Loader, ProtectedRoute, etc.
- src/pages — Home, Products, ProductDetails, Categories, Services, ServiceDetails, Booking, Cart, Checkout, Wishlist, Orders, Bookings, Profile, Addresses, Login, Register, OTP
- src/context — AuthContext, CartContext, WishlistContext
- src/services — api.js, authApi, productApi, serviceApi, orderApi, bookingApi
- src/hooks, src/utils, src/routes, App.jsx

10. Backend Structure

- config, controllers, models, routes, middleware, services, utils, validators, server.js
- Keep controllers thin, move business logic into services, validate all request bodies, and use centralized error handling.
- Use environment variables for database URLs, JWT secrets, payment credentials, cloud storage keys, and OTP/email credentials.

11. UI / UX Direction

- Premium, elegant, luxury, beauty-focused, clean, modern, and trustworthy.
- Use the APK as the visual reference for brand direction, content hierarchy, cards, product/service presentation, and booking concepts.
- Create a proper responsive desktop header/navigation, mobile navigation, footer, breadcrumbs where useful, filters, cards, modals/drawers, loading states, empty states, and error states.
- Responsive targets: 320px+ mobile, 768px+ tablet, 1024px+ desktop, 1440px+ large desktop. No horizontal overflow.

12. Security

- Hash passwords securely.

- Validate and sanitize all inputs.
- Use JWT/session security appropriately; never expose secrets to the frontend.
- OTP must expire and be rate-limited.
- Configure CORS securely.
- Use HTTPS in production.
- Verify payment results server-side.
- Validate booking availability, dates, time slots, service type, artist availability, and pricing on the server.

13. Performance and SEO

- Lazy-load heavy pages/components.
- Optimize and compress images.
- Use pagination for product/service lists.
- Debounce search.
- Use appropriate MongoDB indexes.
- Use API caching/query management where useful.
- Use code splitting.
- Add page titles, meta descriptions, semantic HTML, Open Graph metadata, clean URLs, sitemap/robots configuration, and product structured data where appropriate.

14. Development Workflow for Antigravity

- **Phase 1 — Analyze first:** inspect the APK and create APK_ANALYSIS.md, SITE_MAP.md, DATABASE_SCHEMA.md, API_SPEC.md, COMPONENT_ARCHITECTURE.md, and DEVELOPMENT_PLAN.md.
- **Phase 2 — Foundation:** setup React/Vite/Tailwind/Router, Express, MongoDB, environment configuration, authentication foundation, and shared UI system.
- **Phase 3 — Core UI:** Navbar, Footer, Home, design system, product/category pages.
- **Phase 4 — Shopping:** search, filter, sort, product detail, cart, wishlist, checkout, payment, orders.
- **Phase 5 — Account:** register, OTP, login, profile, addresses.
- **Phase 6 — Beauty booking:** services, service details, artist selection, salon/home choice, date/time, address, summary, confirmation.
- **Phase 7 — Customer history:** orders, bookings, profile, wishlist, addresses.
- **Phase 8 — QA:** responsive testing, API testing, authentication testing, payment-flow testing, booking validation, performance checks, production build.
- After every phase: Implement → Run → Test → Fix → Verify → Continue.

15. Critical Rules

- **NO ADMIN:** Do not implement an admin panel, admin dashboard, admin authentication, admin routes, admin APIs, admin CRUD, or admin-only components.
- Do not invent unrelated features that are not required by the APK/customer product.

- Do not blindly reuse the APK's old backend. Inspect it first; if it is incomplete, insecure, unavailable, or unsuitable, build the required backend cleanly.
- Do not expose API keys, database credentials, JWT secrets, payment secrets, or cloud credentials in frontend code.
- Prefer production-quality reusable code over a one-file prototype.
- Do not proceed blindly with the whole application. Analyze first, then implement phase by phase.

16. Acceptance Criteria

- All major customer-facing APK flows are represented on the website.
- Products can be browsed, searched, filtered, sorted, viewed, added to cart/wishlist, and purchased.
- Authentication and OTP flow works securely.
- Customers can manage profile and addresses.
- Customers can place orders and view order history/details.
- Customers can browse beauty services, select an artist/specialist, select salon or at-home service, choose date/time, provide address where required, and confirm bookings.
- Customers can view booking history/details.
- Website is responsive from mobile to large desktop.
- Production build completes successfully with no blocking console/runtime errors.
- No admin functionality exists anywhere in the website or API.
- Secrets are protected and environment variables are used correctly.

17. Final Antigravity Master Instruction

Build the Nihara D Aura customer-facing website from the uploaded APK. Treat the APK as the primary reference for customer UX, business flows, content concepts, and visual direction. Create a production-ready React/Vite/Tailwind frontend with a Node/Express/MongoDB backend, secure authentication, e-commerce, checkout/payment, and beauty-service booking. **Completely remove the admin component.** There must be no admin dashboard, admin login, admin route, admin API, admin CRUD, or hidden admin functionality. Analyze the APK first, document the architecture, then implement in phases. After each phase, run, test, fix, and verify before continuing. The final result must be responsive, secure, SEO-friendly, performant, and production-build ready.